

AE 06 - Nobels + Sales + Data Import

Ken Hayes

2026-09-25

Contents

Introduction	1
Getting Started	2
Fork and Clone Workflow	2
Packages	2
Part 1: Nobel Winners — CSV Import & Export	2
Understanding File Paths	2
Exercise 1: Read the Data	2
Exercise 2: Split the Data	4
Exercise 3: Write Out the Data	5
Exercise 4: Read the Data Back In	6
Exercise 5: Explore the Subsets	6
Exercise 6: Challenge (Optional)	6
Part 2: Sales — Reading Excel Files	6
Exercise 7: Read the Excel File	7
Exercise 8: Understanding the Data Structure	8
Exercise 9: Reshape to Tidy Format (Stretch Goal)	9
Exercise 10: Additional Data Cleaning (Stretch Goal)	10
Exercise 11: Visualize the Data	10
Exercise 12: Write Out the Tidy Data	11
Exercise 13: Challenge (Optional)	11

Introduction

This application exercise has two parts, both about the same real-world skill: getting messy, real data files into R correctly before you can do anything else with them.

In this application exercise, you will:

- Read CSV files with `read_csv()` and write data back out with `write_csv()`
- Filter data into subsets and understand file paths and data organization
- Read Excel files with `read_excel()`, handling skipped rows and sheets
- Reshape wide data into tidy long format with `pivot_longer()`
- Practice reproducible, well-organized data workflows

Estimated time: 45-60 minutes

Getting Started

Fork and Clone Workflow

Quick reminder: Work in YOUR forked `application-exercises` repository.

1. Navigate to the `ae-06-import-export` folder in YOUR fork
2. Open `ae-06-import-export.Rmd`
3. Update the YAML with your name
4. Verify the `data-raw` folder exists with both `nobel.csv` and `sales.xlsx`

Detailed instructions in AE 01 if needed.

Packages

```
library(tidyverse)
library(readxl)
```

What these packages do:

- **tidyverse:** Data wrangling (`dplyr`), visualization (`ggplot2`), and reading/writing CSVs (`readr`)
 - **readxl:** Reading Excel files (`.xlsx` and `.xls`)
-

Part 1: Nobel Winners — CSV Import & Export

In this part, you'll practice reading data from a CSV file and writing data back out to new files — a common workflow when you need to split a dataset into subsets for further analysis.

Understanding File Paths

Before we start, let's look at the folder structure:

```
ae-06-import-export/
  ae-06-import-export.Rmd  # This file
  data-raw/                # Folder with raw data
    nobel.csv
    sales.xlsx
  data/                   # Folder for processed data (you'll create files here)
```

File paths:

- To read from `data-raw`: `"data-raw/nobel.csv"`
- To write to `data`: `"data/filename.csv"`

Exercise 1: Read the Data

Let's first load the data from the `nobel.csv` file in the `data-raw` folder.

Fill in the blanks:

```
nobel <- read_csv("data-raw/nobel.csv")
```

Hint: The file path is `"data-raw/nobel.csv"`

What `read_csv()` does:

- Reads comma-separated values (CSV) files
- Automatically detects column types
- Returns a tibble (tidyverse data frame)

Verify it worked:

```
# Look at the structure
glimpse(nobel)
```

```
## Rows: 935
## Columns: 26
## $ id <dbl> 1, 2, 3, 4, 5, 6, 6, 8, 9, 10, 11, 12, 13, 14, 1~
## $ firstname <chr> "Wilhelm Conrad", "Hendrik A.", "Pieter", "Henri~
## $ surname <chr> "Röntgen", "Lorentz", "Zeeman", "Becquerel", "Cu~
## $ year <dbl> 1901, 1902, 1902, 1903, 1903, 1903, 1911, 1904, ~
## $ category <chr> "Physics", "Physics", "Physics", "Physics", "Phy~
## $ affiliation <chr> "Munich University", "Leiden University", "Amste~
## $ city <chr> "Munich", "Leiden", "Amsterdam", "Paris", "Paris~
## $ country <chr> "Germany", "Netherlands", "Netherlands", "France~
## $ born_date <date> 1845-03-27, 1853-07-18, 1865-05-25, 1852-12-15,~
## $ died_date <date> 1923-02-10, 1928-02-04, 1943-10-09, 1908-08-25,~
## $ gender <chr> "male", "male", "male", "male", "male", "female"~
## $ born_city <chr> "Remscheid", "Arnhem", "Zonnemaire", "Paris", "P~
## $ born_country <chr> "Germany", "Netherlands", "Netherlands", "France~
## $ born_country_code <chr> "DE", "NL", "NL", "FR", "FR", "PL", "PL", "GB", ~
## $ died_city <chr> "Munich", NA, "Amsterdam", NA, "Paris", "Sallanc~
## $ died_country <chr> "Germany", "Netherlands", "Netherlands", "France~
## $ died_country_code <chr> "DE", "NL", "NL", "FR", "FR", "FR", "FR", "GB", ~
## $ overall_motivation <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ share <dbl> 1, 2, 2, 2, 4, 4, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, ~
## $ motivation <chr> "\"in recognition of the extraordinary services ~
## $ born_country_original <chr> "Prussia (now Germany)", "the Netherlands", "the~
## $ born_city_original <chr> "Lennep (now Remscheid)", "Arnhem", "Zonnemaire"~
## $ died_country_original <chr> "Germany", "the Netherlands", "the Netherlands",~
## $ died_city_original <chr> "Munich", NA, "Amsterdam", NA, "Paris", "Sallanc~
## $ city_original <chr> "Munich", "Leiden", "Amsterdam", "Paris", "Paris~
## $ country_original <chr> "Germany", "the Netherlands", "the Netherlands",~
```

```
# See the first few rows
head(nobel)
```

```
## # A tibble: 6 x 26
##   id firstname surname year category affiliation city country born_date
##   <dbl> <chr> <chr> <dbl> <chr> <chr> <chr> <chr> <date>
## 1 1 Wilhelm Con~ Röntgen 1901 Physics Munich Uni~ Muni~ Germany 1845-03-27
## 2 2 Hendrik A. Lorentz 1902 Physics Leiden Uni~ Leid~ Nether~ 1853-07-18
## 3 3 Pieter Zeeman 1902 Physics Amsterdam ~ Amst~ Nether~ 1865-05-25
## 4 4 Henri Becque~ 1903 Physics École Poly~ Paris France 1852-12-15
## 5 5 Pierre Curie 1903 Physics École muni~ Paris France 1859-05-15
## 6 6 Marie Curie 1903 Physics <NA> <NA> <NA> 1867-11-07
## # i 17 more variables: died_date <date>, gender <chr>, born_city <chr>,
## # born_country <chr>, born_country_code <chr>, died_city <chr>,
## # died_country <chr>, died_country_code <chr>, overall_motivation <chr>,
## # share <dbl>, motivation <chr>, born_country_original <chr>,
## # born_city_original <chr>, died_country_original <chr>,
## # died_city_original <chr>, city_original <chr>, country_original <chr>
```

Task 1.1: How many Nobel laureates are in the dataset?

Your answer: 935_____ laureates

Task 1.2: What variables are in the dataset?

Your answer: firstname, surname, year, category, affiliation, city, country, born_date, died_date, gender, born_city, born_country, and more.

Exercise 2: Split the Data

Nobel prizes are awarded in different categories. Let's split the data into two groups:

- **STEM laureates:** Physics, Chemistry, Medicine
- **Non-STEM laureates:** Literature, Peace, Economics

First, explore the categories:

```
# See what categories exist
nobel %>%
  count(category)
```

```
## # A tibble: 6 x 2
##   category      n
##   <chr>    <int>
## 1 Chemistry  181
## 2 Economics   81
## 3 Literature  114
## 4 Medicine   216
## 5 Peace     133
## 6 Physics    210
```

STEM categories: Physics, Chemistry, Medicine (Physiology or Medicine)

Non-STEM categories: Literature, Peace, Economic Sciences

Task 2.1: Fill in the blanks to create a dataset of STEM laureates.

```
nobel_stem <- nobel %>%
  filter(category %in% c("Physics", "Chemistry", "Medicine"))
```

Hints:

- Use exact category names from the count above
- Medicine might be called "Physiology or Medicine"
- Use %in% to check if category is in a vector of values

How many STEM laureates are there?

```
nrow(nobel_stem)
```

```
## [1] 607
```

Your answer: 607_____ STEM laureates

Task 2.2: Fill in the blanks to create a dataset of non-STEM laureates.

Option 1: List the non-STEM categories explicitly

```
nobel_non_stem <- nobel %>%
  filter(category %in% c("Literature", "Peace", "Economics"))
```

Option 2: Use NOT IN to exclude STEM categories

```
nobel_non_stem <- nobel %>%  
filter(!(category %in% c("Physics", "Chemistry", "Physiology or Medicine")))
```

****Explanation of `!(x %in% y)`:****

- `%in%`: Checks if values are in a vector
- `!`: Negation (NOT)
- Together: "NOT in the list"

****How many non-STEM laureates are there?****

```
` ` ` r  
nrow(nobel_non_stem)  
## [1] 328
```

Your answer: 328__ non-STEM laureates

Verification check:

```
# STEM + non-STEM should equal total  
nrow(nobel_stem) + nrow(nobel_non_stem) == nrow(nobel)
```

```
## [1] TRUE
```

Does this equal TRUE? yes__

Exercise 3: Write Out the Data

Now let's save these two subsets as separate CSV files in the `data` folder.

Create the data folder first (if it doesn't exist):

```
dir.create("data", showWarnings = FALSE)
```

Task 3.1: Write the STEM laureates data to a CSV file.

```
write_csv(nobel_stem, "data/nobel-stem.csv")
```

Hints:

- First argument: the data frame to write (`nobel_stem`)
- Second argument: the file path (`"data/nobel-stem.csv"`)

Task 3.2: Write the non-STEM laureates data to a CSV file.

```
write_csv(nobel_non_stem, "data/nobel-non-stem.csv")
```

Hint: Use a different filename like `"data/nobel-non-stem.csv"`

Verify the files were created:

```
list.files("data")
```

You should see:

- `nobel-stem.csv`
- `nobel-non-stem.csv`

Exercise 4: Read the Data Back In

To verify everything worked, let's read one of the files we just created.

```
stem_check <- read_csv("data/nobel-stem.csv")
nrow(stem_check)
```

```
## [1] 607
```

Does the number of rows match what you expected? Yes__

Exercise 5: Explore the Subsets

Task 5.1: Which STEM category has the most laureates?

```
nobel_stem %>%
  count(category, sort = TRUE)
```

```
## # A tibble: 3 x 2
##   category      n
##   <chr>    <int>
## 1 Medicine    216
## 2 Physics     210
## 3 Chemistry   181
```

Your answer: Medicine has the most STEM laureates.

Task 5.2: Which decade had the most non-STEM laureates?

Hint: You'll need to create a decade variable first.

```
nobel_non_stem %>%
  mutate(decade = (year %/% 10) * 10) %>%
  count(____, sort = TRUE)
```

Your answer: The __1990-2000s had the most non-STEM laureates.

Exercise 6: Challenge (Optional)

Task 6.1: Create a summary that shows the number of laureates by category and gender.

```
nobel %>%
  count(____, ____)%>%
  pivot_wider(names_from = ____, values_from = ____)
```

Task 6.2: Write this summary to a CSV file called `nobel-summary.csv`.

```
# Your code here
```

Part 2: Sales — Reading Excel Files

Excel files are often formatted for humans to read, not for computers to analyze. Common issues include merged cells, multiple header rows, empty rows at the top, data spread across multiple sheets, and inconsis-

tent column names.

Your goal: Read the sales data and reshape it into a tidy format.

Exercise 7: Read the Excel File

First, let's look at what the raw Excel file looks like. Here's a screenshot of the target format:

```
# A tibble: 9 x 2
  id      n
  <chr>   <chr>
1 Brand 1 n
2 1234    8
3 8721    2
4 1822    3
5 Brand 2 n
6 3333    1
7 2156    3
8 3987    6
9 3216    5
```

The data shows sales by brand (rows) and year (columns).

Task 7.1: Read in the Excel file from the `data-raw/` folder.

First, explore the file:

```
# See what sheets are in the file
excel_sheets("data-raw/sales.xlsx")
```

```
## [1] "Sheet1"
```

```
# Read with default settings to see what happens
read_excel("data-raw/sales.xlsx")
```

```
## # A tibble: 5 x 6
##   This file contains information on sales of two~1 ...2 ...3 ...4 ...5 ...6
##   <chr>                                     <chr> <chr> <chr> <chr> <chr>
## 1 Data are organized by brand name, with yearly s~ <NA> <NA> <NA> <NA> <NA>
## 2 <NA>                                           <NA> <NA> <NA> <NA> <NA>
## 3 brand                                           2019 2020 2021 2022 2023
## 4 Brand 1                                           6234 7732 8490 9230 10120
## 5 Brand 2                                           3890 4100 5210 5780 6340
## # i abbreviated name:
## #   1: `This file contains information on sales of two brands across multiple years.`
```

Problems you might notice:

- Empty rows at the top

- Column names might not be in the first row
- Multiple sheets (maybe?)

Task 7.2: Read the file correctly by skipping rows if needed.

```
sales <- read_excel(
  "data-raw/sales.xlsx",          # File path
  skip = 2                        # Number of rows to skip (try 0, 1, 2, 3 and see what works)
)
```

sales

```
## # A tibble: 2 x 6
##   brand   `2019` `2020` `2021` `2022` `2023`
##   <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Brand 1    6234   7732   8490   9230  10120
## 2 Brand 2    3890   4100   5210   5780   6340
```

Hints:

- skip = 3: Skips the first 3 rows
- Try different skip values to see which gives clean column names
- The first column should be brand names

How many rows and columns does the sales data have?

Your answer: 2 rows and 6 columns

Exercise 8: Understanding the Data Structure

```
glimpse(sales)
sales
```

Task 8.1: What does each row represent?

Your answer:

_____The brand and sales

Task 8.2: What does each column represent?

Your answer: Item and sales years _____

Task 8.3: Is this data in tidy format? Why or why not?

Reminder — Tidy data rules:

1. Each variable is a column
2. Each observation is a row
3. Each value is a cell

Your answer:

NO — the data is not tidy. Year values are spread across five separate columns (2019, 2020, 2021, 2022, 2023) instead of being stored in a single year column. This violates the first tidy rule (each variable should be one column) year is one variable, but here it's split across multiple columns.

Hint: In tidy format, “year” should be a single column, not multiple columns.

Exercise 9: Reshape to Tidy Format (Stretch Goal)

The goal is to reshape the data to look like this:

```
# A tibble: 7 x 3
  brand    id     n
  <chr>  <dbl> <dbl>
1 Brand 1  1234     8
2 Brand 1  8721     2
3 Brand 1  1822     3
4 Brand 2  3333     1
5 Brand 2  2156     3
6 Brand 2  3987     6
7 Brand 2  3216     5
```

This means:

- One column for `brand`
- One column for `year`
- One column for `sales` (the values)

Task 9.1: Use `pivot_longer()` to reshape the data.

```
sales_tidy <- sales %>%
  pivot_longer(
    cols = -brand,          # Which columns to pivot (all except brand)
    names_to = "year",      # Name for the new column with old column names
    values_to = "sales"     # Name for the new column with values
  )

sales_tidy
```

```
## # A tibble: 10 x 3
##   brand  year sales
##   <chr> <chr> <dbl>
## 1 Brand 1 2019  6234
## 2 Brand 1 2020  7732
## 3 Brand 1 2021  8490
## 4 Brand 1 2022  9230
## 5 Brand 1 2023 10120
## 6 Brand 2 2019  3890
## 7 Brand 2 2020  4100
## 8 Brand 2 2021  5210
## 9 Brand 2 2022  5780
## 10 Brand 2 2023  6340
```

Hints:

- `cols = -brand`: All columns except `brand`
- `names_to = "year"`: Creates a column called “year”
- `values_to = "sales"`: Creates a column called “sales”

What `pivot_longer()` does:

- Takes wide data and makes it long
- Column names become values in a new column
- Values from those columns go into another new column

How many rows does the tidy data have?

```
nrow(sales_tidy)
```

Your answer: 10__ rows

Why does this number make sense?

Your answer: Original: 2 brands $\times$ 5 years = 10 rows in tidy format

Exercise 10: Additional Data Cleaning (Stretch Goal)

Task 10.1: Convert the year column to numeric.

Why? After pivoting, year is stored as character (text), but it should be numeric. “sales_tidy <- sales_tidy %>% mutate(year = as.numeric(year))

```
class(sales_tidy$year)
```

****Task 10.2:**** Remove any rows with missing sales values.

```
`sales_tidy <- sales_tidy %>%
  filter(!is.na(sales))
```

```
nrow(sales_tidy)
```

Task 10.3: Arrange the data by brand and year.

```
sales_tidy <- sales_tidy %>%
  arrange(brand, year)
```

```
sales_tidy
```

```
## # A tibble: 10 x 3
##   brand  year sales
##   <chr> <chr> <dbl>
## 1 Brand 1 2019  6234
## 2 Brand 1 2020  7732
## 3 Brand 1 2021  8490
## 4 Brand 1 2022  9230
## 5 Brand 1 2023 10120
## 6 Brand 2 2019  3890
## 7 Brand 2 2020  4100
## 8 Brand 2 2021  5210
## 9 Brand 2 2022  5780
## 10 Brand 2 2023  6340
```

Exercise 11: Visualize the Data

Now that the data is tidy, we can easily create visualizations!

```
ggplot(sales_tidy, aes(x = year, y = sales, color = brand)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
```

```
labs(
  title = "Sales by Brand Over Time",
  x = "Year",
  y = "Sales",
  color = "Brand"
) +
theme_minimal()
```

Task 11.1: Which brand has the highest sales in the most recent year?

Your answer: Brand 1__

Task 11.2: Which brand shows the most growth over time?

Your answer:

Brand 1: $6,234 \rightarrow 10,120 = +3,886$ _Brand 2: $3,890 \rightarrow 6,340 = +2,450$ _____

Exercise 12: Write Out the Tidy Data

Save the tidy data to a CSV file for future use.

```
dir.create("data", showWarnings = FALSE)

write_csv(sales_tidy, "data/sales-tidy.csv")
```

Hint: Use a filename like "data/sales-tidy.csv"

Exercise 13: Challenge (Optional)

Task 13.1: Read the sales data from a different sheet (if the Excel file has multiple sheets).

```
excel_sheets("data-raw/sales.xlsx")

# sales_sheet2 <- read_excel("data-raw/sales.xlsx", sheet = 1)
```

Task 13.2: Calculate the total sales for each brand across all years.

```
sales_tidy %>%
  group_by(brand) %>%
  summarise(total_sales = sum(sales)) %>%
  arrange(desc(total_sales))
```

Reflection

****What did you learn about file organization and data workflows from the Nobel exercise?****

__Working through this exercise reinforced how much file paths depend on where code is actually running

****Why is it useful to split data into subsets?****

_Splitting data into subsets lets you analyze and compare groups separately instead of mixing categories

```

**What was the most challenging part of reading and cleaning the Excel data?**
Figuring out the correct number of rows to skip (skip = 2) was the trickiest part, the raw file had blank rows
-----

**Why is tidy data important for analysis and visualization?**

_Tidy data makes it possible to plug data into tools like ggplot2 and dplyr without extra reshaping, so you can focus on the analysis.
-----

**What's one thing you learned about `pivot_longer()`?**

_pivot_longer() converts wide data into long format by collapsing those columns into two by one holding the values.
---

# Submitting Your Work

**Before you finish:**

1. Make sure all code chunks run without errors
2. Fill in all blanks and answer all questions
3. Check that CSV files were created in the `data` folder
4. Knit to HTML first to verify everything works

## Git Workflow

1. Click the **Git** pane
2. Check boxes next to all changed files, including:
   - `ae-06-import-export.Rmd`
   - `data/nobel-stem.csv`
   - `data/nobel-non-stem.csv`
   - `data/sales-tidy.csv` (if you completed the stretch goals)
3. Click **Commit**
4. Write a commit message: "Completed AE 06 - Nobels + Sales + Data Import"
5. Click **Commit**
6. Click **Push**
7. Verify your work is on GitHub in **YOUR fork**

**Important:** Commit BOTH your code AND the CSV files you created!

## Generate PDF for Canvas Submission

Once everything works in HTML:

1. Click the **Knit** dropdown arrow
2. Select **Knit to PDF**
3. Wait for PDF generation
4. Find `ae-06-import-export.pdf` in the Files pane
5. Export and save to your computer
6. Submit the PDF to Canvas

**Remember:** Push your `.Rmd` to GitHub (your checkpoint/backup) AND submit the PDF to Canvas. **The PDF is the final submission.

```

Troubleshooting

CSV reading/writing issues:

- ****Error: "cannot open file":**** Check your file path - make sure ``data-raw/nobel.csv`` exists
- ****Error when writing:**** The ``data`` folder might not exist - run the ``dir.create()`` code
- ****File paths on Windows:**** Use forward slashes ``/`` not backslashes ``\``
- ****Can't find created files:**** Look in the ``data`` folder in the Files pane
- ****`%in%` not working:**** Make sure values match exactly (check spelling and capitalization) - use ``cour``

Excel reading issues:

- ****Error: "file does not exist":**** Check the file path ``data-raw/sales.xlsx``
- ****Column names are weird:**** Try different ``skip`` values
- ****Extra empty columns:**** Normal for Excel files - they'll be removed during cleaning
- ****Can't install readxl:**** Make sure you've loaded the package: ``library(readxl)``

Pivot issues:

- ****Too many/too few rows after pivot:**** Check your ``cols`` argument
- ****Column names not what you expected:**** Check ``names_to`` and ``values_to``
- ****Year is character not numeric:**** Use ``mutate(year = as.numeric(year))``

Visualization issues:

- ****Plot doesn't show:**** Make sure data is tidy first
- ****Too many lines:**** Check that you completed the pivot correctly
- ****Can't see all brands:**** Try adjusting colors or using ``facet_wrap(~brand)``

If you're stuck:

- Review the walkthrough video
- Check documentation: ``?read_csv``, ``?read_excel``, ``?excel_sheets``, ``?pivot_longer``
- Ask on the discussion board
- Attend office hours

Quick Reference

Function	Package	Use
<code>`read_csv()`</code>	<code>readr</code>	Read a comma-separated values file into a tibble
<code>`write_csv()`</code>	<code>readr</code>	Write a data frame out to a CSV file
<code>`read_excel()`</code>	<code>readxl</code>	Read an Excel file, with options for sheet, skip, and column names
<code>`excel_sheets()`</code>	<code>readxl</code>	List all sheet names in an Excel file
<code>`pivot_longer()`</code>	<code>tidyr</code>	Reshape wide data into long/tidy format
<code>`dir.create()`</code>	base R	Create a new folder (won't error if it already exists)
<code>`list.files()`</code>	base R	List files in a folder
<code>`file.exists()`</code>	base R	Check if a file exists